

ISMILE 2026 · PROJECT BRIEF

Backend, database, payments & launch plan

A practical build plan for turning the current iSmile website into a secure registration and ticketing system, with Pretix and bank payments.

Recommended architecture: keep the current frontend; use Django for the custom backend, PostgreSQL for its database, Pretix for ticket orders and capacity, and a protected dashboard for staff workflows.

Prepared 23 September 2026

Planning document · Decisions about banks and Pretix hosting require confirmation before implementation.

1. Current state and target

The website already has pages for registration, sponsorship, speakers, workshops, schedules, partners, and content editing. Registration collects personal information, professional or student ticket choice, university details, an optional student-ID image, and a FIB or FastPay choice. It supports English, Arabic, and Kurdish.

The critical gap: the registration page currently shows a success message and creates a reference in the browser, but it does not save an attendee, create an order, or collect payment. Sponsor enquiries currently open WhatsApp or email. The current content admin is useful, but its browser-only access fallback cannot protect attendee or payment information.

The release goal is a complete journey: submit information, review student eligibility where needed, reserve a ticket, pay through a supported bank, verify the result on the server, issue the ticket, and give staff a reliable view of every step.

2. System design

Component	Responsibility
Existing frontend	Public information, accessible forms, language support, status screens.
Django backend	Server validation, staff access, student review, sponsor workflow, integration, reports.
PostgreSQL	Attendees, custom registrations, review decisions, sponsor records, integration state, audit history.
Pretix	Ticket products, quotas, orders, ticket issuance, check-in, authoritative ticket status.
Bank providers	Charge and refund money through approved APIs or Pretix payment plugins.
Private file storage	Student-ID uploads and other sensitive documents.
Background worker + Redis	Notifications, retries, reconciliation, email and long-running tasks.
Email service	Registration updates, approval notices, and support messages.

Pretix should own ticket inventory and order state. Our database stores the custom information and references to Pretix records. If Pretix is self-hosted, it gets separate database credentials; the custom backend accesses it through its authenticated API.

3. Database plan

Record	Main fields and purpose
Event	Name, dates, venue, timezone, registration settings.
Attendee	Internal ID, name parts, email, normalized phone, preferred language.
Registration	Attendee, event, specialty, ticket type, submitted answers, status, timestamps.
Student review	University, private file reference, decision, reviewer, reason and dates.
Ambassador code	Code, owner, event, validity, attribution and approved discount policy.
Pretix link	Organizer, event, product, order, position and ticket IDs.

Payment attempt	Provider, exact amount, currency, external reference, state and verified result.
Sponsor enquiry	Company, contact, requested package or booth, assignee, notes and status.
Content and staff	Speakers, schedules, workshops, translations, roles and accounts.
Operations	Received webhook IDs, background jobs, audit events and message delivery.

Use database relationships and unique constraints to prevent duplicate external transactions and broken links. Represent money exactly with currency; use consistent timestamps and display event times in Asia/Baghdad. Store files privately, with only file references in the database. Keep a person separate from their registration so the system can support later events.

4. Registration and payment flow

Form → server validation → student review (if required) → Pretix order and reservation → bank payment → server verification → Pretix ticket → staff dashboard

1. The browser sends form data and the image to the backend. The backend validates fields, upload type and size, and saves a registration. It returns a server-generated reference.
2. If a student needs approval, staff review the private document. On approval, the student can continue to purchase. The initial policy should avoid charging before approval.
3. The backend creates or starts the Pretix order using current ticket price and availability. The browser never decides the authoritative price or whether an order is paid.
4. The customer completes the provider flow. A browser redirect shows a pending status until the server verifies the payment with the provider or Pretix.
5. Pretix issues the ticket only after verified payment. Webhooks trigger synchronization; the backend fetches current Pretix data through the authenticated API before updating its local view.
6. A scheduled task checks unresolved records and reconciles them with Pretix and the bank. Failed email delivery must not invalidate a paid order.

Submission and payment operations need idempotency keys: repeating a request must not create another order or charge. The workflow must cover timeouts, repeated or delayed callbacks, expired reservations, cancellations, full or partial refunds where supported, and the customer closing the page.

5. Pretix and bank integration

Pretix manages orders and tickets; a bank or wallet processes money. A bank API alone does not guarantee a ready Pretix integration. For each provider, check whether a compatible Pretix payment plugin already exists. If not, assess whether the chosen Pretix hosting permits a custom plugin. Build and fully test one provider before adding another.

For every bank we need its current documentation, sandbox credentials, allowed currency, hosted payment or checkout flow, callback and verification rules, refund capabilities, and settlement reports. Keep API secrets on the server. Prefer the provider's hosted payment interface so iSmile does not handle card details or bank credentials.

Pretix documents both payment-provider plugins and webhooks. Its webhook documentation warns that notifications can be duplicated and their contents should be used only as a trigger to fetch current data through the authenticated API. This is the behavior our integration will implement.

6. Dashboard and daily operations

Use Pretix's own management screens for ticket operations. Build a protected Django staff dashboard for iSmile-specific work. Begin with Django admin for internal records and add focused custom screens for tasks that require a clearer workflow.

Area	Staff actions
Overview	See paid orders, pending reviews, failed synchronization and enquiries.
Attendees	Search, filter, inspect, correct permitted details and export when authorized.
Student approvals	Inspect private documents, approve or reject, record reason and reviewer.
Finance	Inspect verified payment state, reconcile issues and make controlled refunds.
Sponsors	Assign enquiries, add notes and follow up on packages or booths.
Content	Edit speakers, program, partners, images and translations.

Define owner, registration, finance, content, and check-in roles. Enforce permissions on the server for every action. Record approvals, exports, refunds and important changes in an audit history. The current browser-only admin fallback must be removed or disconnected from private data in production.

7. Security, privacy and recovery

- Require HTTPS, secure sessions, multi-factor authentication for staff, and rate limits for login and public forms.
- Validate every field again on the server. Use safe file processing, file-size limits and real file-type checks.
- Keep student documents private; issue short-lived access links only to authorized reviewers. Set retention and deletion rules.
- Keep bank credentials, Pretix tokens and application secrets in server-side secret storage; keep them out of browser code and the repository.
- Protect browser actions from forged requests. Keep logs free of passwords, payment secrets and unnecessary personal details.
- Back up PostgreSQL and private files off the application server. Test a real restore. Monitor backup failures, payment mismatches and worker failures.

Collect only information that the event team actually needs. Confirm the purpose of age, gender and student-ID collection before launch. Limit staff access and decide how long each category of data is retained.

8. Domain and hosting

Buy the domain when its name and ownership are settled. Choose server capacity after confirming whether Pretix is hosted or self-hosted. A preferred setup serves the public site through a CDN, runs Django and workers on a Linux application server, uses managed PostgreSQL and private object storage, and keeps a separate staging environment.

A single VPS can work for a constrained budget, but it is a shared failure point. Keep off-server backups and separate staging. A provisional starting size for the custom backend, with database and Pretix hosted elsewhere, is 2–4 virtual CPUs and 4–8 GB RAM; actual sizing depends on peak demand and testing. Do not expose PostgreSQL or Redis publicly.

Budget for the domain, hosting, database, private storage, backups, email delivery, monitoring, Pretix costs if applicable, payment fees, and ongoing maintenance.

9. Build sequence and acceptance criteria

Phase	Work	Done when
1. Rules	Confirm ticket types, prices, currency, approval policy, refunds, workshop capacities and expected demand.	Rules and data fields are written down.
2. Payment proof	Inspect Pretix hosting and connect one bank in a sandbox.	A verified test payment reaches the correct Pretix order.
3. Foundation	Set up Django, PostgreSQL, staff login, staging, deployment and backups.	Protected backend runs in staging.
4. Forms	Save registrations, private student files and sponsor enquiries; add student review.	Accepted submissions persist and can be reviewed.
5. Ticketing	Map ticket types and workshops into Pretix; implement reservation and expiry.	Concurrent bookings do not oversell.
6. Payments	Finish callbacks, verification, retries, reconciliation and refunds; add other banks.	Success, failure and repeat scenarios pass.
7. Staff tools	Build role-based views, exports, content workflows and audit history.	Staff can perform daily work with proper access.
8. Launch	Review security, load, backup restore, mobile checkout and event-day support.	Controlled live transactions and ticket delivery pass.

Before public launch, test simultaneous buyers for the last seat; duplicate callbacks; delayed or missing payment notices; expired and refunded tickets; access to another attendee's record; private student images; and restoration from backup. Train check-in staff and define a connectivity fallback.

10. Decisions needed to finalize implementation

1. **Pretix:** pretix.eu, another hosted service, or self-hosted? Provide the URL and version without credentials.

2. **Payments:** confirm all banks and wallets, starting with FIB and FastPay if these remain the choices. Provide current API documentation links first, not live secrets.
3. **Scale:** expected total attendees and expected simultaneous registrations at peak.
4. **Budget and deadline:** monthly hosting budget, launch date, and who will maintain the server.
5. **Ticket policy:** ticket prices and currency, workshop limits, refunds, and whether students need approval before paying.

First implementation milestone: complete one sandbox purchase through Pretix and one bank, including a verified order status. That result will settle the payment architecture and inform the final hosting choice.

References

Pretix payment guide: docs.pretix.eu/guides/payment/

Pretix payment-provider development: docs.pretix.eu/dev/development/api/payment.html

Pretix webhooks: docs.pretix.eu/dev/api/webhooks.html

Django admin: docs.djangoproject.com/en/5.2/ref/contrib/admin/

Django deployment checklist: docs.djangoproject.com/en/6.1/howto/deployment/checklist/

PostgreSQL backups: postgresql.org/docs/current/backup.html